

Task 1:

TASK 1: Implement a producer-consumer communication system using anonymous pipes where the parent process generates data and the child process consumes it. Measure the communication efficient Code:

```
rohita@LaxmiDurga: ~  
GNU nano 8.7.1  
#include <stdio.h>  
#include <stdlib.h>  
#include <unistd.h>  
#include <sys/wait.h>  
#include <time.h>  
#include <string.h>  
  
#define DATA_SIZE 100000  
  
int main()  
{  
    int pipefd[2];  
    pid_t pid;  
    char buffer[1024];  
    char data[] = "Producer sends data to Consumer through pipe.\n";  
    long total_bytes = 0;  
  
    struct timespec start, end;  
  
    if (pipe(pipefd) == -1)  
    {  
        perror("pipe");  
        exit(1);  
    }  
  
    pid = fork();  
  
    if (pid < 0)  
    {  
        perror("fork");  
        exit(1);  
    }  
  
    if (pid == 0)  
    {  
        // Child - Consumer  
        close(pipefd[1]);  
  
        while (1)  
        {  
            int n = read(pipefd[0], buffer, sizeof(buffer));  
  
            if (n <= 0)  
                break;  
  
            total_bytes += n;  
        }  
  
        close(pipefd[0]);  
  
        printf("Consumer received %ld bytes.\n", total_bytes);  
        exit(0);  
    }  
    else  
    {  
        // Parent - Producer  
        close(pipefd[0]);  
  
        clock_gettime(CLOCK_MONOTONIC, &start);  
  
        for (int i = 0; i < DATA_SIZE; i++)  
        {
```

```

rohita@LaxmiDurga: ~
GNU nano 8.7.1
}
if (pid == 0)
{
    // Child - Consumer
    close(pipefd[1]);

    while (1)
    {
        int n = read(pipefd[0], buffer, sizeof(buffer));

        if (n <= 0)
            break;

        total_bytes += n;
    }

    close(pipefd[0]);

    printf("Consumer received %ld bytes.\n", total_bytes);
    exit(0);
}
else
{
    // Parent - Producer
    close(pipefd[0]);

    clock_gettime(CLOCK_MONOTONIC, &start);

    for (int i = 0; i < DATA_SIZE; i++)
    {
        write(pipefd[1], data, strlen(data));
    }

    close(pipefd[1]);

    wait(NULL);

    clock_gettime(CLOCK_MONOTONIC, &end);

    double time_taken =
        (end.tv_sec - start.tv_sec) +
        (end.tv_nsec - start.tv_nsec) / 1e9;

    long bytes_sent = DATA_SIZE * strlen(data);

    printf("Producer sent %ld bytes.\n", bytes_sent);
    printf("Communication time: %.6f seconds\n", time_taken);

    if (time_taken > 0)
    {
        double throughput =
            (bytes_sent / time_taken) / (1024 * 1024);

        printf("Communication throughput: %.2f MB/s\n",
            throughput);
    }
}

return 0;
}

```

Output:

```

gopinath@GOPINATH:~$ gcc w5.c -o w5
gopinath@GOPINATH:~$ ./w5
Consumer received 4600000 bytes.
Producer sent 4600000 bytes.
Communication time: 0.147341 seconds
Communication throughput: 29.77 MB/s
gopinath@GOPINATH:~$ █

```

Observation: The parent process produces data and writes it into the anonymous pipe, while the child process reads and consumes the data. Communication efficiency is measured using the time taken and data throughput (MB/s).

Task 2: Develop a program that executes the equivalent of the shell command: `ls -l grep` using `fork()`, `pipe()`, `dup2()`, and `exec()` system calls.

Code:

```
rohita@LaxmiDurga: ~  
GNU nano 8.7.1  
#include <stdio.h>  
#include <stdlib.h>  
#include <unistd.h>  
#include <sys/wait.h>  
  
int main()  
{  
    int pipefd[2];  
    pid_t pid1, pid2;  
  
    // Create pipe  
    if (pipe(pipefd) == -1)  
    {  
        perror("pipe");  
        exit(1);  
    }  
  
    // Create first child for ls -l  
    pid1 = fork();  
  
    if (pid1 == -1)  
    {  
        perror("fork");  
        exit(1);  
    }  
  
    if (pid1 == 0)  
    {  
        // Redirect stdout to pipe  
        dup2(pipefd[1], STDOUT_FILENO);  
  
        close(pipefd[0]);  
        close(pipefd[1]);  
  
        // Execute ls -l  
        execlp("ls", "ls", "-l", (char *)NULL);  
  
        perror("execlp ls");  
        exit(1);  
    }  
  
    // Create second child for grep  
    pid2 = fork();  
  
    if (pid2 == -1)  
    {  
        perror("fork");  
        exit(1);  
    }  
  
    if (pid2 == 0)  
    {  
        // Redirect stdin from pipe  
        dup2(pipefd[0], STDIN_FILENO);  
  
        close(pipefd[0]);  
        close(pipefd[1]);  
  
        // Execute grep .  
        execlp("grep", "grep", ".", (char *)NULL);  
  
        perror("execlp grep");  
        exit(1);  
    }  
  
    // Parent closes both ends of pipe  
    close(pipefd[0]);  
    close(pipefd[1]);  
  
    // Wait for both children  
    waitpid(pid1, NULL, 0);  
    waitpid(pid2, NULL, 0);  
  
    printf("Parent: Both processes completed.\n");  
  
    return 0;  
}
```

Output:

```
gopinath@GOPINATH:~$ gcc w5i.c
gopinath@GOPINATH:~$ ./a.out
total 268
-rw-r--r-- 1 gopinath gopinath 57 Aug 6 14:47 First.c
drwxr-xr-x 3 gopinath gopinath 4096 Aug 3 05:02 hello-world
drwxr-xr-x 11 gopinath gopinath 4096 Jul 30 09:39 OSPP
-rw-r--r-- 1 gopinath gopinath 1322 Aug 25 03:28 p-week2.c
-rw-r--r-- 1 gopinath gopinath 1206 Aug 7 15:18 Practical2.c
-rw-r--r-- 1 gopinath gopinath 935 Aug 6 15:00 Skill2.c
-rw-r--r-- 1 gopinath gopinath 654 Aug 7 14:32 Skill3.c
-rw-rw-r-- 1 gopinath gopinath 802 Aug 7 14:48 Skill5Task2.c
-rwxr-xr-x 1 gopinath gopinath 16356 Aug 27 09:50 a.out
-rw-r--r-- 1 gopinath gopinath 936 Aug 5 16:22 command_exec.c
-rw-r--r-- 1 gopinath gopinath 0 Aug 25 04:47 copy.txt
-rw-r--r-- 1 gopinath gopinath 24 Jul 25 04:56 destination.txt
-rw-r--r-- 1 gopinath gopinath 1 Jul 28 14:48 nano-506.save
-rw-r--r-- 1 gopinath gopinath 729 Aug 25 04:43 new.c
-rw-rw-r-- 1 gopinath gopinath 270 Aug 25 04:46 new1.c
-rwxr-xr-x 1 gopinath gopinath 16216 Aug 25 04:50 newi2
-rwxr-xr-x 1 gopinath gopinath 1252 Aug 25 04:55 newi2.c
-rw-r--r-- 1 gopinath gopinath 710 Aug 25 04:42 os-pr2.c
-rw-r--r-- 1 gopinath gopinath 931 Aug 28 08:27 os.c
-rw-r--r-- 1 gopinath gopinath 156 Aug 18 07:07 p.c
drwxr-xr-x 6 gopinath gopinath 4096 Aug 4 08:12 OSPP
-rw-r--r-- 1 gopinath gopinath 148 Aug 19 09:39 s1.c
-rw-r--r-- 1 gopinath gopinath 881 Aug 19 23:47 s12.c
-rw-r--r-- 1 gopinath gopinath 559 Aug 19 23:57 s2.c
-rw-r--r-- 1 gopinath gopinath 538 Aug 20 00:00 s21.c
-rw-r--r-- 1 gopinath gopinath 75 Aug 25 05:01 sample.txt
-rw-r--r-- 1 gopinath gopinath 3735 Aug 22 06:33 sk1.c
-rw-r--r-- 1 gopinath gopinath 1908 Aug 11 07:08 skillTask1.c
-rw-r--r-- 1 gopinath gopinath 968 Aug 7 14:52 skill3Task2.c
-rw-r--r-- 1 gopinath gopinath 5428 Aug 20 06:36 skillweek2.c
-rw-r--r-- 1 gopinath gopinath 1287 Aug 20 09:05 skillweek4.c
-rw-r--r-- 1 gopinath gopinath 4502 Aug 20 06:39 skillweek41.c
-rw-r--r-- 1 gopinath gopinath 20 Aug 25 05:38 sourc2.c
-rw-r--r-- 1 gopinath gopinath 1885 Aug 20 05:34 sweek2.c.save
-rw-r--r-- 1 gopinath gopinath 1855 Aug 26 05:29 w-p2.c
-rwxr-xr-x 1 gopinath gopinath 16424 Aug 27 09:45 w5
-rwxr-xr-x 1 gopinath gopinath 1771 Aug 27 09:42 w5.c
-rwxr-xr-x 1 gopinath gopinath 16336 Aug 27 09:49 W51
-rw-r--r-- 1 gopinath gopinath 1305 Aug 27 09:48 w51.c
-rw-r--r-- 1 gopinath gopinath 3010 Aug 24 14:05 wTask22.c
-rw-r--r-- 1 gopinath gopinath 1472 Aug 12 06:12 weeeK4.c
-rw-r--r-- 1 gopinath gopinath 2926 Aug 24 13:41 weeeKTask4.c
-rw-r--r-- 1 gopinath gopinath 920 Aug 27 08:09 week4.c
-rwxr-xr-x 1 gopinath gopinath 16256 Aug 27 08:37 week42.c
-rw-r--r-- 1 gopinath gopinath 811 Aug 27 08:19 week42.c
-rw-r--r-- 1 gopinath gopinath 2688 Aug 26 06:19 week4Task1.c
-rw-r--r-- 1 gopinath gopinath 926 Aug 27 08:14 week41.c
Parent: Both processes completed.
gopinath@GOPINATH:~$
```

Observation:

- The program successfully implements the `ls -l | grep` command using `fork()`, `pipe()`, `dup2()`, and `exec()`.
- It demonstrates **inter-process communication**, where the output of `ls` is passed through the pipe to `grep`.